



Some Reflections on Strachey and His Work

DANA SCOTT

dana.scott@cs.cmu.edu

School of Computer Science, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15213, USA

Christopher Strachey and I first met at an IFIP WG2.2 meeting in Vienna in the summer of 1969. This was the first one I attended. These were lively meetings with lots of arguments. I was immediately struck by Strachey's striving to isolate clear-cut general principles, and, though I did not understand everything he said, I found his approach the most sympathetic of the members of the group. I had spent a year on sabbatical in Amsterdam and had started a collaboration with Jaco De Bakker. Having been introduced to ALGOL 58/60 in the early 60s in Berkeley, and having taught some graduate courses in the emerging computer-science environment at Stanford, I was eager to go more deeply into programming-language semantics. As I was just accepting a call to Princeton, I had to make special arrangements to have my first semester on leave to work with Strachey at Oxford. Christopher provided me a visiting membership in the new Wolfson College at Oxford, and a very intense period of work ensued.

Using some of the material from my 1977 introduction to Joe Stoy's textbook on denotational semantics [12], I would like to think back over the influence of Christopher Strachey and my too brief association with him.

His untimely death at the age of 58 in May, 1975, left a permanent void in the lives of his relatives, colleagues, students and many friends, academic and non-academic. In the ten years he worked in Oxford, where he had held a personal chair since 1971, the field of computing grew and changed remarkably, in no little measure owing to his own many original ideas and his influence on the work of others. This was generously acknowledged when the British Computer Society made him one of its first Distinguished Fellows in 1972.

Strachey's own publications were few, however, though that was not always his fault. (And the present volume helps to remedy the situation in part.) In the last year before his brief, final illness Strachey laboured on the Milne-Strachey book, since he saw the need for a comprehensive account of the approach to programming-language semantics he advocated. A preliminary manuscript was completed in 1974, but he was unable to start on the necessary revisions for publication. The book published in 1976 by Robert Milne under their earlier title *A Theory of Programming Language Semantics* [5] was completely rewritten by Milne and contains in a very substantial portion his own important and original contributions, which developed as an outgrowth of his earlier Cambridge Ph.D. work done (at Oxford) under Strachey's direction. It is not being unfair to Milne to say that the published book is not what Strachey would have written: their styles of exposition were just very, very different, and Milne had hardly any other option than to start over in order to produce a consistent and complete text. It proved to be not really suitable as an introduction to the method, however, and many, more expository books have been written since then. Strachey

had hoped that the earlier manuscript would have qualified for a special Adams Essay Prize at Cambridge, and he was sadly disappointed in his very last days at the hospital to learn that it had been refused.

To me, in any case, it is doubtful whether Strachey could have written a satisfactory beginning text himself. His lecturing style was by turns crisp, muddled, cryptic, delightful, maddening, sarcastic, always highly spontaneous, but—to those that had ears—unfailingly stimulating. What he could have done, and did do on several occasions, was to produce shorter pieces which would, with great clarity, put forward what were the basic ideas and important problems, since his unique experience gave him special insight. True, he had his prejudices and hobby-horses, but many of us could, and did, forgive him everything for the sake of his excellent ideas. This is not meant to imply that he was incapable of sustained work, since that is far from the truth, but academic writing was a considerable chore for him and in a certain sense was a waste of his time. Of course, no one had any idea just how limited his time was, so we were doubly grateful that Joe Stoy took the responsibility as a long-time assistant and coworker with Strachey to set down not only what he learnt from him, and others, but what he developed himself for exposition over a considerable period of lecturing at Oxford and during a sabbatical year at MIT. Stoy's book helped very much to explain the methodology to researchers. Again this may not have been exactly the book Strachey would have wanted, but it served to complement Milne and Strachey's book very well indeed.

It should also not be thought that Milne and Stoy were the only workers in this direction of language definition at the time. References to many other names can be found in the bibliographies of these two books, and in the papers to which they in turn refer, as well as in the references in the current volume. Furthermore, the other well-known approaches, in particular the Vienna Definition Method, the Algol 68 method, and the axiomatic method, all had influence on Strachey's ideas. Sometimes he disagreed strongly with certain features of the other approaches, but he took full account of what they were doing—though his approach was quite different. It should also not be thought that Strachey was not concerned with the details of the semantics, even though he did not write so much himself. In the first place during many protracted periods we would all talk things over with him almost daily. In the second place, he lectured every year at Oxford, and many of the formulations and examples people still use today are his, though it might be difficult to trace back to the exact origin of every point. Finally, he could be very critical and emphatic about exactness and clarity, so that sometimes proposals went through many versions. It is difficult, therefore, to give precise credit to him, but a large group of people are still glad to say how great a debt they feel.

Language definition was not by any means the only interest of Strachey and his group. His field was *computing* in a dynamic sense: how to get the machine to do what *you* want. Thus, a programming language is obviously a necessary tool. Strachey felt that people had made design mistakes on the language level that were as bad as those they had made on the machine or operating-system level, and he was concerned with improving rationality at all levels. Personally I think that one of our greatest losses in his death is that his design philosophy could not become more widely known. Philosophy in design of machines (or algorithms or languages) is something that has to be learned by doing it. Strachey should have been allowed much greater scope to develop and pass on his ideas; but we can be

thankful that in his last decade he was given a group at all, first by the Science Research Council and later as taken over by Oxford University—though at a reduced level. His ideas and philosophy continue to live on, however, in the work of his students and associates. After his death, his personal chair at Oxford was made permanent, and the first holder of this professorship, C.A.R. Hoare, has just retired after directing an excellent growth of teaching and research in many different aspects of computing science. Times change, of course, and different people have different interests, but I feel that that Strachey could be happy that so much he established eventually took root.

To understand the context of Strachey's work, I feel we should review Strachey's somewhat unorthodox early career.¹ On 16 November 1916 Christopher Strachey was born into the middle of the Bloomsbury set, with complicated connections on both sides of his remarkable family. His father was Oliver Strachey, who was an older brother of Lytton Strachey and who had a career not in literature but in cryptography. From his father Christopher perhaps took his lifelong interest in recreational puzzles. His mother was Ray Costelloe, a strong woman from a line of strong women, and perhaps from her he inherited his own special independence of mind. From Gresham's School he went up to King's College as a Mathematical Scholar in 1935. His undergraduate career was academically undistinguished, and Strachey never took an advanced degree. Graduating just before the war, he worked throughout it as a physicist in the Valve Research Laboratories of Standard Telephone and Cables Ltd., where experience with a differential analyser awakened his interest in mathematical machines.

Surprisingly, he became a schoolmaster after the war, first at St. Edmund's Canterbury (1945–1949) and then at Harrow (1949–1951). It was a very important experience in his development, however, since it affected his whole approach to teaching and making people understand things. Once in a great while I found him acting rather too much the schoolmaster, but I was always envious of the magician's way he had of capturing a person's attention and interest. At Oxford, for example, he had particular success with vacation courses in computing for undergraduates, and he was also excellent with teaching pupils from (pre-university) schools about computing. In an oblique way one might say computing can be like playing a musical instrument, and talent shows itself early. (Strachey himself was also a keen amateur musician.) But in those days the standard school curriculum provided little opportunity for music and even less for computing. Certainly the "New Maths" (and I am afraid, some later reforms) proved to have been a failure in this direction, probably by being too abstract. Here, in computing for schools, is another line where Strachey's special qualifications and ideas remained to too great an extent unrecognized and unfulfilled.

On the other hand, we should note that Strachey was quite opposed to "Computing Science" as an undergraduate specialist course of studies *in itself* (that is, according to the British pattern, where a student would study nothing else at the university). At that age students—he felt—should learn serious things that they will need later, rather than the latest "state-of-the-art" mish-mash which will be out of date before they graduate. At least that was his opinion, and perhaps he was right in some respects; but surely *some* serious technical education is needed, and a mix of required subjects and computing options might have been the answer—both in a mathematics curriculum and in engineering. I remember from Strachey's remarks to me that the extremely slow development of his subject at Oxford was deeply discouraging to him in his last few years. Times change, however, and the

importance and influence of computing now at the turn of the century differs profoundly from what it was in 1975. Most major universities—including Oxford—now have very popular degree programs. Unfortunately, in many countries now, interest in *pure mathematics* has alarmingly diminished. Personally, I would like to see mathematics and computing becoming more jointly supportive in the future, but that is a topic for another essay.

Returning to Strachey's personal history, suddenly in 1950, while still a schoolmaster and an unknown amateur, Strachey appeared at the National Physical Laboratory with a draughts-playing program for their Pilot ACE. His next big achievement, a simulator for the Manchester Mark I, was far larger than any previous program for that machine, and this achievement firmly established his position in the computing field in Britain.

In 1951 he joined the National Research and Development Corporation, where he was responsible for the overall design for the Ferranti Pegasus computer, which successfully embodied his belief that ease of programming should guide the design of machines. Seconded to Toronto, he programmed most of a backwater simulation for the St. Lawrence Seaway. This was another *tour de force*, and advanced the opening of the Great Lakes to shipping by many years. Later, along with one or two others, he became interested in the use of a computer for several *concurrent* tasks; indeed, it was he who published the earliest paper on this [13], though he attached little significance to the fact.

From 1959 Strachey was a freelance consultant (somewhat unusual in those days), and his business even prospered sufficiently to employ an assistant (Peter Landin—see other reminiscences in this volume). He was living in London then, and he formed a select discussion group, which met monthly at his home, for people interested in the then new field of “automatic programming”. In 1962 he gave up his private consulting and went to work at the Mathematical Laboratory in Cambridge, where he was also a Senior Research Fellow of Churchill College.

About Cambridge, David Barron writes:

“At the time we were designing the hardware and software of the Titan computer (later known as Atlas 2), and I was in charge of the University software team. One day I was summoned to [Professor M.V.] Wilkes's office and told that Strachey was going to join my team. I was staggered by the idea that Christopher should be working for me, and indeed I might as well have tried to hold a (friendly) tiger by the tail. In fact that is a fairly apt simile. We were supposed to be designing a language and writing a compiler. Christopher set off, and the rest of us were pulled somewhat breathlessly behind. We never did get a compiler, but at the end of it he had formulated some of the fundamental ideas of modern programming languages, had sown the seed for the work on formal semantics that he and Peter Landin were later to pursue, and I had gained an understanding of programming languages the depth of which I have only recently come to appreciate.”

Concerning some of those fundamental ideas, Strachey wrote:

“Algol 60, surely one of the most successful pieces of original work by a committee, had already introduced some important ideas, including the concept of scope. Without experience it is not possible to be certain, but it is generally felt that John McCarthy

played a large part in this. At the time, McCarthy was working on Lisp which is closely based on Church's λ -calculus.² The λ -calculus has been widely used as an aid in examining the semantics of programming languages precisely because it brings out very clearly the connection between a name and the entity it denotes, even in cases where the same name is used for more than one purpose. The earliest suggestion that the λ -calculus might be useful in this way that has come to our notice was in a verbal communication from Roger Penrose to [me] in about 1958. At the time this suggestion fell on stony ground and had virtually no influence."

David Barron continues:

"We devoted a lot of time in the early days [at Cambridge] to syntactic matters, particularly to the problem of defining operator precedences so that the 'obvious' mathematical meaning of an expression is retained. However, this preoccupation with syntax was short lived, being terminated by the enunciation of Strachey's first law of programming: *Decide what you want to say before you worry about how you are going to say it.*"

On the rôle of syntax, Strachey himself said:

"The syntactic problem was the first to be recognised and, with the introduction and general acceptance of BNF, the first to receive a satisfactory solution. The problems of syntax are much easier than those of semantics. This had the natural but unfortunate consequence that great attention was given to syntactic questions—so much so, indeed, as to give rise to the idea that syntax and grammar were the whole of a programming language. Nothing could be further from the truth as the widespread and continuing misunderstandings about even the most basic concepts, such as names and values, illustrates vividly."

On his work at Cambridge, Strachey also commented:

"From about 1962 until 1966 [I] was engaged with several collaborators at Cambridge and London on the design of a general purpose programming language which came to be known as CPL (an acronym which stood for so many different titles in the course of its existence that it finally acquired a life of its own). During the course of this design work a rather large number of concepts became clear, including many of those concerned with the store such as the distinctions between L- and R-modes of evaluation and between scope and extent. Though both these distinctions were originally due to myself they were refined in further discussions. CPL was never satisfactorily implemented and only inadequately publicised; the most complete document concerning it was only circulated privately (although quite widely)."

For a more detailed view of the history of CPL, the reader should consult the paper of Campbell-Kelly cited earlier, as well as the book of Wilkes, *Memoirs of a Computer*

Pioneer [16]. Professor Maurice Wilkes is indeed a “computer pioneer” who has been at the forefront of many important developments—including putting the first fully operational stored-program computer, the first EDSAC, on the air in 1949, a truly historic event that is perhaps not given enough credit in the United States. Some years later, when involved in a subsequent project, Wilkes comments in [16] (pp. 224 ff.):

“When the Titan project was launched we had to consider what high-level languages we would provide to go with our shiny new computer. The idea that languages should be independent of machines had been given strong impetus by the ALGOL movement and by FORTRAN, which had originally been developed by IBM for the 704 and was rapidly becoming an industry-wide language. It was clear that FORTRAN would be required for the Titan, and equally clear that the Cambridge Autocode, which had been written by David Hartley for EDSAC 2 and which had proved a great success, would also be required. We were anxious to provide some more modern language as well. In retrospect, it is clear that we should have provided ALGOL 60. However, while recognizing that the development of this language was an important advance, I was ambivalent about it as a language for serious computation.”

I believe Wilkes was right on both counts: ALGOL 60 (or perhaps the later version ALGOL W of Wirth) would have been the right choice for them and for ALGOL—which was in a way defeated both by the entrenchment of FORTRAN in scientific computing (still alive and kicking today) and by the IBM effort to construct and push PL/I (now dead). But in 1960 it was not at all clear that the necessary run-time efficiency could be provided for ALGOL-like languages. If they had worked on this problem, they might have been able to solve it. Wilkes, after explaining his reservations, continues:

“My colleagues felt much as I did on these issues and accordingly we decided to try our hand at defining and implementing a new language to be based on ALGOL 60 but free from the defects of that language as we saw them. The name chosen for the new language was CPL which originally stood for Cambridge Programming Language but which later, when a group at London University joined in the project, was re-interpreted to mean Combined Programming Language. *This project was a complete failure.*³ The original goal of producing a language suitable for practical use was lost sight of and the project became one of research into language design and implementation.”

From an historical perspective, I believe it is debatable that the work on CPL was a *complete* failure, though the primary objective could not be met. In 1985, and even today, as I know from direct contact and from reports of others, Professor Wilkes still feels betrayed both personally and politically by what happened to this project. He continues in his book:

“I felt much mortification over the failure of the CPL project, and my feeling of failure was accentuated when some years later, during a visit to Stanford University, I had experience of ALGOL W, a language designed by Niklaus Wirth. This I perceived to be exactly the kind of development of ALGOL 60 that I was hoping for as a result

of the CPL project. However, I began to look at the matter very differently when I read Tony Hoare's Turing Lecture delivered in 1980. Hoare described a computer-language project he had been associated with at Elliot Brothers Ltd. at about the same time as we were struggling with CPL. Although on a larger scale, this project ran into very similar difficulties and eventually had to be abandoned. Other contemporaneous software projects also ran into grave difficulties, although in some cases the difficulties were eventually surmounted. The fact was that we were all over-ambitious and wholly lacking in experience of large software projects. We did not appreciate how many and how seductive were the opportunities for escalation, and having no insight into the dangers could establish no defenses."

I think the point is that the project was very much ahead of its time. But, of course, without ambition there is no progress. With hindsight, a more evolutionary approach to language design and implementation would have been more prudent. On the other hand, I am not sure with the state of experience at the time there could have been the "defenses" Wilkes asks for. He continues:

"The fundamental causes of the failure of the CPL project were those I have just indicated. However the immediate cause was my appointment to the staff of Christopher Strachey. At the time that the Manchester Mark I computer was coming into action, Strachey was earning his living as a school master. Through a slight acquaintanceship with Turing he had an opportunity to try his hand at programming for it and his abilities as a programmer soon became apparent. He was fortunate enough to attract the attention of Lord Halsbury, then head of the National Research and Development Corporation, a British Government agency charged with providing finance for promising technological projects, and he joined the staff of the Corporation. One of the projects being supported was the development of the Ferranti Pegasus computer. As a representative of the Corporation, Strachey exerted a strong influence on the design of this computer, of which a number were in due course made and sold. At the time I invited him to come to Cambridge he was practising as a private consultant in London. He and I shared a number of views on the subject of ALGOL 60 and we had collaborated in a paper that was published in the communications of the Association for Computing Machinery.

I liked Strachey personally, but I realized that he was not an easy man to work with, and that I was taking a chance in employing him. He was rather overbearing in technical discussions and apt to insist on minor points with the same force that he insisted on major ones. For example, he did not like the use of the ALGOL word **else**, which he said was ungrammatical, and he insisted that in our paper we should use **otherwise**. I protested that there were more important issues to raise than that one, but he was adamant.

Coming to Cambridge had greatly improved Strachey's academic standing and he would have liked to stay there and lead a group to work on language theory. Halsbury, who was then Chairman of the relevant committee of the Science Research Council, was anxious to provide financial backing. Strachey, however, wanted much more

freedom than he could have had within the Cambridge University framework as it then existed, and in the end he set up a group in Oxford. It was there that he finally got his act together. He met Dana Scott, and their work began to attract attention. It was too theoretical for me to assess and I found that none of my more mathematical friends would commit themselves as to its value. In fact, it was not until after Strachey's sudden and tragic death in 1975, as a result of a liver infection, that the work he had done with Scott achieved general recognition."

To say the least, this is a very condensed review of Strachey's career. Fortunately the Campbell-Kelly biographical note and the memoirs of this volume will let readers fill in other details and evaluations and to judge the matter for themselves. When Michael Rabin and I were jointly awarded the Turing Award in 1976, it was not just for our early paper on finite automata but also for our subsequent work. Similarly, Wilkes's Turing Award in 1967 was not just for his work in the '40s but also for his continuing influence and multiple contributions.

Many people feel—despite the checkered history—that the influence of CPL was quite remarkable. For example, an implementation of a variant of CPL, devised by Strachey's student Martin Richards and called BCPL (not mentioned in Wilkes's book) established itself as an important implementation language in its own right [7]. BCPL was used to very good advantage by Strachey later at Oxford when he developed the operating system for his Modular One machine at his Programming Research Group.

After leaving Cambridge and before coming to Oxford in 1965 to set up his group, Strachey spent an influential year at MIT. 1964 saw the advent of two crucial papers: Landin's paper on the mechanical evaluation of expressions [3], and Strachey's *Towards a Formal Semantics* [14] (published in 1966 but delivered at a working conference in 1964). Strachey comments:

"Although Landin's influence on the developments of our semantic method has been considerable, it has been largely indirect. We prefer in the first instance to use a more abstract approach and the standard semantics differs from the work of Landin in that it does not depend in any way on a machine involving anonymous quantities and linkage information not explicitly available to the programmer."

And further: "[Paper [14]] introduced the general method of giving mathematical semantics as a set of recursively defined functions from syntactic domains to semantic domains which included higher-order functions. The approach was deliberately informal, and, as subsequent events proved, gravely lacking in rigour—but, in spite of these defects, it certainly laid down the outline of our subsequent work."

The kind of work Landin did (and others, as in the Vienna method [4],) comes to the fore again in describing precisely how to *implement* a language. This is discussed in Chapter 13 of Stoy's book [12] and, in great detail, in Milne and Strachey's book [5]. The distinction drawn between language *definition* and language *implementation* is a vital point in understanding Strachey's approach. I think the distinction is regarded as obvious today.

The fullest discussion on the first phase of Strachey's work on the conceptual basis of programming languages was contained in a series of lectures given by him at the International

Summer School in Computer Programming at Copenhagen in August 1967. Strachey's lecture notes [15] were privately circulated, but unfortunately the editor did not publish the proceedings of the meeting. The essential content was incorporated into the first draft of Milne and Strachey's book [5], but it is only now that the full text can be published.

Concerning the basic ideas of this paper, Milne and Strachey remarked in the introduction to the draft of their book [5] that there were four features of mathematical semantics as it now stands which were not in the 1964 paper and which they did not consider their own work. They said:

“The first is that the store, σ , is a function from locations to their contents, following a suggestion made by Rod Burstall to [Strachey] in 1964. Before then we had made no detailed model of the store and had merely assumed the existence of certain basic functions operating on it. The construction of a proper model helped to bring out the fact that storing commands (or procedures) led to severe mathematical problems.”

Perhaps the last word on the modeling of storage has not been said even yet—and this has been a serious weakness in the denotational approach—and the reader is referred to the discussion and references in Tennent's contribution to this volume for current thinking and the improvements that have been proposed.

The second divergence they mention is the separation of the *store* from the *environment*. This was a suggestion of mine made in fall of 1969 when our joint work was just starting. The idea was common from logic and the semantics of the “static” languages employed in the theory of models as understood by logicians. The distinction has become standard in semantical discussions.

The third contribution in their list was my own construction of λ -calculus models and more complicated reflexive domains in November of 1969. I have written about this discovery, and Strachey's influence, in my Turing Lecture [8]. The main point is to have an *extensional* theory of functions, related directly to computability, where functions (rather than syntactic representations or programs for the functions) can be treated as objects in themselves. More background on the idea can be found in the several papers of mine (and in the joint paper with Strachey [10]). Over the intervening years, many people have taken up this idea for a variety of purposes, and it is becoming a flexible tool for a theory of computability on abstract domains with some real applications. The current state of work is very well documented in two recent textbooks [1, 11].

The final piece of the puzzle mentioned by Milne and Strachey is the introduction of *continuations* to deal with the complicated problem of jumps and error exits from procedures. This was due to Christopher Wadsworth, and independently Lockwood Morris, and is an application of an idea of A. Mazurkiewicz in a paper of 1970. The reader is referred to the contribution by Wadsworth in this volume and to John Reynolds's earlier paper on the discoveries of continuations [6].

Milne and Strachey also said:

“There is another facet to the development of mathematical rigour in the discussion of programming languages which is perhaps more technically mathematical than most of those so far discussed. This is the development of *induction rules* and the closely

related *fixed-point operators*. The relevant theorems about fixed points, due to Knaster and Tarski, had little impact on computing [as distinct from recursive function theory] until the work of David Park and Scott. Landin in his 1964 paper made use of the ‘paradoxical combinator’, Y , discussed by Curry and Feys. In λ -calculus Y has the formal properties of a fixed-point operator, the definition of which requires the application of a function to itself. After Scott had completed his theory of reflexive domains, Park showed that this operator could be identified with the [minimal fixed-point function]. The paradoxical combinator was extensively used in the early stages of mathematical semantics as a minimal fixpoint operator without any satisfactory mathematical justification; fortunately, subsequent events showed that these uses were in fact substantially correct. The development of induction rules for mathematical semantics was started by McCarthy, who introduced ‘recursion induction’. This rule turned out to be inadequate; more satisfactory rules were discussed by de Bakker and Scott.”

The point here is that McCarthy’s domains of computation were too limited, and the rôle of continuity of functions and inclusivity of predicates was not apparent. When the better domains were introduced, better induction rules were required. Much more has to be done in developing practical proof techniques, and many people are working on this problem at the present time.⁴

The semester at Oxford in the autumn of 1969 was a very intense time, working with Strachey, meeting Peter Landin, David Park, John Reynolds, and many others. As I recounted in my Turing Lecture, I did not intend at all to construct models for the type-free λ -calculus. In trying to give a mathematically clean and abstract foundation for the kind of semantics Strachey envisioned, it suddenly became clear to me that there were more structures with suitable properties than I think anyone had imagined before. The idea eventually led to “domains” with far more structure than what was needed for a mathematical semantics of pure λ -calculus.

In January of 1970 I took up my post at Princeton University. Strachey came for a visit the next year, and our joint work continued. In 1972, out of the blue, I received a call to become the first Professor of Mathematical Logic at Oxford—a new chair that had just been established. I do not think it was Strachey’s influence that resulted in this offer. It was a difficult decision to decide to go to Oxford after having been so short a time at Princeton. However, the computer-science environment was not at that time very well developed at Princeton, and in any case I was a professor of Philosophy there. The many colleagues I knew in Oxford in many areas and the hope to continue joint work with Strachey convinced me to accept. Alas, in the event, both Strachey and I had so many obligations and duties as new professors at Oxford that our joint work could never again be so concentrated. There were good relations with many people in the Programming Research Group, but Strachey’s commitment to writing the Adams Prize Essay—in which by the rules from Cambridge I could not join—took up a very great deal of his attention. His very unfortunate death then put an end to any future plans. I regretted very much being excluded from joint work at the time, but the Adams Prize meant extremely much to Strachey: his two periods at Cambridge (as a student and as a researcher) had not been successes. He wanted very much to show

Cambridge as an institution that he could make lasting scientific contributions. Too bad that the work was in too early a stage and that death was waiting around the corner.

In effect Strachey lived to see the completion of the second main phase of the work on mathematical semantics. In 1977 I said:

“For the future the problems of an adequate proof theory and of explaining non-determinism loom very large. In evaluating Strachey’s own contribution we can at a very minimum say that he *founded* a subject, and with his inspiration in mind we can now set about making it useful.”

The many references in this volume and the papers they refer to show that progress has been steady and substantial. Many new methods have been introduced and new insights have been secured. But Strachey’s inspiration can yet have more influence. I still look forward to the future.

Appendix: One last historical comment

The report I wrote in 1969 for Strachey—and which was widely circulated in typescript—emphasizing the type-theoretical version of λ -calculus and least fixed points was finally published with commentary in 1993 in a festschrift for Corrado Böhm [9].

Notes

1. Much of the information that follows was taken either from the *Times* obituary of 23 May 1975, from a personal reminiscence by Professor David Barron published in the *Computer Bulletin* for September 1975, from the King’s College Cambridge obituary, or from the historical preface written for the Milne-Strachey manuscript, but not employed by Milne in the published version [5]. Since then, the extensive biographical note by Martin Campbell-Kelly, published in 1985 [2], has described Strachey’s life and work in much greater detail than is possible here.
2. We have to note, however, despite what Strachey said, that McCarthy denies that Church’s calculus directly influenced LISP.
3. Emphasis added.
4. This was true in 1977 and is still true today despite much progress.

References

1. Amadio, R.M. and Curien, P.-L. *Domains and Lambda-Calculi*. Cambridge University Press, 1998.
2. Campbell-Kelly, M. Christopher Strachey, 1916–1975, A Biographical Note. *Annals of the History of Computing* 7(1) (1985) 19–42.
3. Landin, P.J. The mechanical evaluation of expressions. *Computer Journal* 6 (1964) 308–320.
4. Lucas, P. and Walk, K. On the formal description of PL/I. *Annual Review in Automatic Programming* 6 (1969) 105–152.
5. Milne, R.E. and Strachey, C. *A Theory of Programming Language Semantics*. Chapman and Hall, London, and John Wiley, New York, 1976.
6. Reynolds, J.C. The discoveries of continuations. *Lisp and Symbolic Computation* 6(3/4) (1993) 233–247.

7. Richards, M. BCPL: A tool for compiler writing and system programming. In *Proceedings AFIPS Spring Joint Computer Conference*, Boston, Mass., American Federation of Information Processing Societies, May 1969, pp. 557–566.
8. Scott, D. Logic and programming languages. *Communications of the ACM* **20** (1977) 634–641.
9. Scott, D. A type-theoretical alternative to ISWIM, CUCH, OWHY. *Theoretical Computer Science* **121** (1993) 411–440.
10. Scott, D.S. and Strachey, C. Towards a mathematical semantics for computer languages. In *Proceedings of the Symposium on Computers and Automata*, Polytechnic Institute of Brooklyn, 1971.
11. Stoltenberg-Hansen, V., Lindström, I., and Griffor, E.R. *Mathematical Theory of Domains*. Cambridge University Press, 1994.
12. Stoy, J.E. *Denotational Semantics: The Scott-Strachey Approach to Programming Language Theory*. The MIT Press, 1977.
13. Strachey, C. Systems analysis and programming. *Scientific American* **25**(3) (1966) 112–124.
14. Strachey, C. Towards a formal semantics. In *Formal Language Description Languages for Computer Programming*, T.B. Steel, Jr. (Ed.). North-Holland, 1966, pp. 198–220.
15. Strachey, C. Fundamental concepts in programming languages. In *Proceedings of the 1967 International Summer School in Computer Programming*, Copenhagen, Denmark, 1967. Unpublished lecture notes.
16. Wilkes, M.V. *Memoirs of a Computer Pioneer*. The MIT Press, 1985.